

Appendix: Referenced Artifacts

This appendix concatenates the full content of every artifact directly referenced by [System Requirements Specification: biz.dfch.SpecMgr] (sysrs-8d752304-b076-4bad-89af-f8032158dd21), plus every artifact those artifacts themselves reference in their own ## Related Artifacts sections (none introduced any artifact beyond the set already listed directly). Generated manually per [GitHub issue #86] (<https://github.com/dfch/biz.dfch.SpecMgr/issues/86>): each artifact's YAML frontmatter is rendered as a fenced yaml code block (not literal frontmatter) so multiple documents can be concatenated safely into this one file, followed by the artifact's body verbatim.

- 2 Goal (GOL) documents
- 8 Architecture Decision Record (ADR) documents
- 14 Requirement (REQ) documents

24 artifacts total, in the same order they are cross-referenced from the SysRS body (Goals, then Decisions, then Requirements grouped by ISO/IEC 25010:2023 characteristic).

GOL 08666592-a2d2-4309-95c6-3c94248ca342

```
classification: null
created: '2026-09-03 10:25:10.866+02:00'
id: 08666592-a2d2-4309-95c6-3c94248ca342
status: draft
type: gol
updated: '2026-09-03 10:25:10.866+02:00'
version: 1.0.0
```

AI-Agent-Native Specification Artifact Management

THE project shall provide an MCP server that AI agents and other MCP clients can use to create, read, list, update, and validate structured specification artifacts across the full requirements-engineering document lifecycle, so that specification work stays machine-readable and consistently structured for the agents performing it.

Description

The project's own README.md states its purpose as "An artifact manager for system specifications" and describes itself as "an MCP server that you can use to manage different specification artifacts", listing thirteen already-implemented artifact types (ADR, DEC, FEAT, GOL, PRB, QA, REQ, RSK, SOP, SYSRS, TSK, UC, VCR). AGENTS.md's Status section confirms each of these types is backed by its own schema-validated domain package exposing create/read/list/validate MCP tools. This goal captures that founding, organization-wide purpose, not any single domain package's behavior.

Source

README.md ("An artifact manager for system specifications" / "This project is an MCP server that you can use to manage different specification artifacts") and AGENTS.md's Status section (the domain-package inventory).

Notes

Captured retrospectively during feat-84-specmgr-sysrs (GitHub issue #84) while drafting this repository's own System Requirements Specification.

GOL b663528e-08c5-426b-9f20-32192c0a3bdb

```
classification: null
created: '2026-09-03 10:25:14.254+02:00'
id: b663528e-08c5-426b-9f20-32192c0a3bdb
status: draft
type: gol
updated: '2026-09-03 10:25:14.254+02:00'
version: 1.0.0
```

Cross-Referenceable, Non-Duplicating Specification Artifacts

THE project shall keep every specification artifact type schema-validated and addressable by a stable id, so that higher-level documents such as a System Requirements Specification can aggregate existing goal, problem-statement, question-and-answer, use-case, requirement, risk, decision, and verification artifacts by cross-reference rather than by duplicating their content.

Description

AGENTS.md describes the domain-first architecture underlying every document-type package: each domain owns its own schema, and cross-cutting generic tools (update/set_status/set_classification/delete) operate uniformly across domains instead of duplicating per-domain logic. The sysrs domain package (feat-32-sysrs) is built specifically around this idea: its sections accept only type-tagged cross-reference bullets (e.g. GOL <uuid>: <title>, REQ <uuid>: <title>) pointing at existing documents, never inline copies of their content. This goal states the underlying design intent that makes that aggregation possible in the first place.

Source

AGENTS.md's description of the domain-first architecture (ADR ece4554b-725c-4f76-bc04-5d2b760363d2) and the sysrs domain's cross-reference-only design (.specmgr/feat/feat-32-sysrs/README.md).

Notes

Captured retrospectively during feat-84-specmgr-sysrs (GitHub issue #84) while drafting this repository's own System Requirements Specification.

ADR ece4554b-725c-4f76-bc04-5d2b760363d2

```
status: accepted
decision-makers: dfch
id: ece4554b-725c-4f76-bc04-5d2b760363d2
version: 1.0.0
```

Organize the codebase by document-type domain: domain-first hierarchy for tools/ prompts/resources, shared versioned models

Context and Problem Statement

As the SpecMgr project grows to support multiple document types (initially ADR, later req, uc, ac, etc.), the code organization must scale. Two patterns exist: (1) interface-layer-first (top-level tools/, prompts/, resources/ packages, each with domain sub-packages like tools/adr/, prompts/adr/), or (2) domain-first (top-level adr/, req/, uc/ packages, each containing its own tools/, prompts/, resources/ sub-packages). Interface-layer-first scatters each domain across three locations; domain-first co-locates all code for one document type. The schema layer (Pydantic models, parser, renderer, mutations) must also have a clear home.

Decision Drivers

Maintainability and discoverability: developers working on ADR functionality should find all ADR code in one place; future document types (req, uc, ac) should follow the same pattern; schema mutations (update_section, option_create, etc.) need a consistent, domain-agnostic location; schema versioning must support long-term evolution (v1, v2, etc.) without code duplication.

Considered Options

Interface-layer-first (tools/adr/, prompts/adr/, resources/adr_*) vs. domain-first (adr/tools/, adr/prompts/, adr/resources/); mutations as Pydantic model methods vs. pure free functions; full wholesale vN duplication per major schema version vs. minimal diff (only changed classes in vN, import unchanged classes from vN-1).

Decision Outcome

Adopt a domain-first hierarchy. Create top-level domain packages (adr/, and later req/, uc/, ac/) each containing tools/, prompts/, resources/ sub-packages. Keep models/ as a shared, top-level package organized internally by domain (models/adr/, models/req/, etc.) with major-version sub-packages (models/adr/v1/, models/adr/v2/, etc.). Mutations (update_section, set_status, option_*, etc.) live as pure free functions in models/adr/v1/mutations.py (not as Pydantic model methods), taking whole Adr objects and returning new Adr objects or read-only data, never mutating their arguments. For major schema version upgrades, create a minimal vN package containing only the classes that changed; unchanged classes are imported from vN-1. This avoids a full N-parsers-times-N-renderers maintenance matrix and prevents drift from unrelated bugfixes.

Consequences

All ADR-related code is co-located under adr/; future domains (req/, uc/, ac/) will follow the identical structure, making the codebase predictable and maintainable. Shared models/ reduces duplication and keeps schema validation centralized. Free-function mutations are stateless and composable. Minimal vN duplication (only changed classes) keeps migration code simple and maintainable. Trade-off: domain packages are now top-level, adding one level of nesting compared to interface-layer-first; developers unfamiliar with the convention must learn the pattern once.

Confirmation

Verify that adr/, models/adr/, tests/adr/, tests/models/adr/ follow the documented structure; verify that mutations are pure functions with no side effects; verify no Pydantic model classes define their own mutation methods.

ADR 33c5ab08-ff58-4c73-8c32-23abaf3838e3

```
status: accepted
decision-makers: dfch
id: 33c5ab08-ff58-4c73-8c32-23abaf3838e3
version: 1.0.0
```

Filesystem is the sole source of truth: no in-memory id-to-document cache

Context and Problem Statement

The MCP server is a long-running process. When the LLM or a human edits an ADR file directly (outside the MCP tools), the server's in-memory state becomes stale. The question is: should the MCP server maintain an in-memory cache of parsed ADR objects keyed by id, or should it always re-read from disk?

Decision Drivers

Correctness under concurrent hand-edits and LLM-driven edits; simplicity and predictability (no cache invalidation logic); no staleness problems.

Considered Options

Server-side cache (id → parsed ADR object, shared across tool calls) vs. re-read/re-parse/re-render/re-write on every tool call.

Decision Outcome

The .md file on disk is the sole source of truth. Every MCP tool call re-reads the current on-disk state, re-parses it via the schema's parser, applies the mutation through in-memory Pydantic models, validates the result, re-renders the full file deterministically, and writes the file back. No in-memory cache of parsed documents is maintained. This ensures that hand-edits by users always see the latest state, and concurrent edits are safe (each tool call gets a fresh read of the current file).

Consequences

Guaranteed correctness under concurrent edits (human and LLM). No cache invalidation logic needed. Trade-off: every tool call includes I/O and parsing overhead. At expected ADR-repo scale (dozens to low hundreds of files), this overhead is acceptable; the filesystem scan is cheap enough.

More Information

File I/O and parsing are centralized in `adr/tools/_io.py` (`read_adr`, `write_adr`, `load_by_id`) and `adr/tools/_paths.py` (`find_adr_path` for id-to-path resolution via fresh directory scan).

ADR 8cf940c5-3100-485c-a12d-14b59b631712

```
status: accepted
decision-makers: dfch
id: 8cf940c5-3100-485c-a12d-14b59b631712
version: 1.0.0
```

id/filename/addressing scheme: server-generated UUID, {id}-{slug}.md, directory-scan resolution

Context and Problem Statement

ADR files must be addressable by the MCP tools via a unique identifier. The filename must be both human-readable and stable. Questions: (1) what format should the id be (sequential counter, UUID, etc.)? (2) what is the filename format? (3) how does the server resolve an id to a file path (cached index, directory scan, etc.)? (4) where is the ADR base directory?

Decision Drivers

Human-readable filenames; unique, stable ids that don't require external state (counters); id resolution that works even after concurrent file edits; configurable base directory for different deployment contexts.

Considered Options

Sequential counter filenames (0001-..., 0002-...) vs. UUID-prefixed filenames ({uuid}-{slug}.md); cached id-to-path index vs. fresh directory scan; CLI argument vs. environment variable for base-directory config.

Decision Outcome

Use server-generated UUID strings as the `id` field (created once by `create_adr`, never reassigned). Filename format is `{id}-{slug}.md` where `slug` is derived from the ADR title at creation time. id-to-path resolution is performed via fresh directory scan + frontmatter parse on every call (no cached index), ensuring correctness under concurrent hand-edits. The ADR base directory is configurable via the `SPECMGR_ADR_DIR` environment variable (default `./docs/adr`). Option numbering within a single ADR uses a monotonically increasing, never-reused counter (not the id); deleting an option leaves a gap.

Consequences

Filenames are human-readable and slug-based (e.g., abc123-use-madr-4-0-0.md). No external counter state required; UUIDs are globally unique. Fresh directory scans on every lookup guarantee correctness even after concurrent edits. Environment-variable config allows flexible deployment (different base directories for different test runs, CI, production). Trade-off: slightly longer filenames due to UUID prefix; directory scans are more expensive than O(1) index lookups, but acceptable at expected scale.

More Information

id-to-path resolution is implemented in `adr/tools/_paths.py`: `find_adr_path(id)`, `iter_adr_paths()`, `adr_base_dir()`. Option numbering is managed by `models/adr/v1/mutations.py` (`option_create`).

ADR 832cd6c1-ef8a-4bfc-990e-a610823f61ae

```
status: accepted
decision-makers: dfch
id: 832cd6c1-ef8a-4bfc-990e-a610823f61ae
version: 1.5.0
```

Generic heading-mapped markdown-to-Pydantic parsing with declarative Heading metadata and opt-in constraints

Context and Problem Statement

SpecMgr's only existing document type (ADR) has a hand-rolled parser/renderer pair (`models/adr/v1/parser.py`, `renderer.py`) tailored to its fixed heading layout plus a dynamic Option collection (see ADR 4c6119c9d5). As additional document types are introduced (req, uc, ac, ...), each with their own heading structure, frontmatter shape, and nesting depth (e.g. a use-case document's `## Characteristic Information` section itself contains a dozen `###` sub-sections such as Goal in Context, Scope, Preconditions), writing a bespoke hand-rolled parser per document type does not scale and duplicates the same token-walking logic repeatedly. A generic engine is needed that maps markdown headings to typed Pydantic fields declaratively, supports arbitrary recursive nesting (not just one heading level), and lets a section's allowed content (tags, length, prose constraints) be validated without hand-writing a bespoke `model_validator` for every model.

Decision Drivers

Reusability across current and future document types without duplicating token-walking code per type; support for arbitrary heading-nesting depth, since real documents (e.g. use cases) nest

sections at least two levels deep; minimizing per-model boilerplate for structural validation; round-trip fidelity for inline formatting (strong/emph) inside a heading itself must be preserved without a separate metadata-driven heading-synthesis step; avoiding a byte-exact round-trip requirement by default for body content, since markdown has many equally valid renderings (list bullet style, wrapping) of the same semantic content and a strict default would be fragile; keeping the new engine independent of the existing ADR-specific parser so the ADR pipeline is not disturbed; robustness against silently misassigning an out-of-order or omitted optional section's content to the wrong field.

Considered Options

Option 1a (superseded): declarative `Annotated[MarkdownStr, Heading(tag=, alias=)]` field metadata plus a single generic recursive parser/renderer engine, with the heading line itself resynthesized at render time from `tag+alias` metadata rather than stored tokens. Option 1b (adopted): encode heading level structurally via a fixed `MarkdownHeading1..MarkdownHeading6` base-class hierarchy (one class per heading depth), each carrying a default "no heading at-or-above my own level among my nested tokens" invariant validator; heading identity (alias) is declared once at the class level (defaulting to a Title-Case derivation of the class name when omitted), not repeated per field via an `Annotated` wrapper; every heading-bearing instance stores its own `heading_open/inline/heading_close` token triple verbatim (not resynthesized), so rendering is uniform (`render(self._tokens)`) and inline formatting inside a heading round-trips for free. Option 2: an imperative class-level decorator that registers heading mappings on a model class. Option 3: convention-only matching, deriving the expected heading text purely from the Python field name with no explicit per-field/per-class metadata.

Decision Outcome

Adopt Option 1b. Add a new package, `models/md/` (not `models/markdown/v1/` as originally sketched here -- implementation settled on this shorter path, with no separate `v1/` version subfolder unlike `models/adr/v1/`), with no dependency on `models/adr/v1/`; the existing ADR parser/renderer keep their hand-rolled implementation per ADR 4c6119c9-532f-4629-8977-108e78304f48 and are not migrated onto this engine as part of this decision.

`MarkdownStr` (`markdown_str.py`) is the base value object. It does **not** retain parsed `markdown-it-py` tokens on the instance (`_tokens`, as first decided here) -- implementation settled on a private `_value: str` attribute holding the already-rendered, `mdformat-normalized` source text instead, with `str()/__repr__` returning `_value` unchanged for a leaf class (no nested `MarkdownStr` fields) or the normalized concatenation of every nested field's own `str()` for a composite class. This still delivers this decision's core promise -- inline formatting inside a heading round-trips for free, with no separate metadata-driven heading-synthesis step -- just via stored rendered text rather than replayed tokens.

Heading-level structure is still encoded via a fixed class hierarchy, but under the names settled on during implementation: an abstract `MarkdownSection` (`markdown_section.py`) and six concrete leaves, `MarkdownSection1` through `MarkdownSection6` (`markdown_section1.py..markdown_section6.py`) -- not `MarkdownHeading1..MarkdownHeading6` as originally sketched here. Each `MarkdownSectionN` just pins `@markdown(type="heading_open", tag="hN")` (see below). The "no nested same-or-higher-level heading" invariant this decision originally assigned to a dedicated `model_validator` per class is instead enforced procedurally by `MarkdownSection.get_extent` itself (it stops at any sibling/ancestor heading, i.e. level $\leq$ its own,

folding a nested *deeper* heading into the current section's extent) plus `MarkdownStr.from_text`'s mandatory trailing-completeness check (`remaining_text == ""`); a separate `model_validator(mode="after")` per class was drafted during implementation (`markdown_section.py`'s `validate_heading_structure`, `markdown_section1.py..6.py`'s `validate_headings`) but its assertions are currently commented out/inert -- a known follow-up, not a silently-passing check.

Heading identity/matching, which this decision originally sketched as a single placeholder annotation (`# @some_annotation(alias="...")`, "to be formalized as a real class attribute/decorator during implementation"), was formalized as **two** independent class-level decorators rather than one: - `@markdown(type=, tag=)` (`markdown.py`) attaches structural metadata (the markdown-it token type and HTML tag) -- this is what `MarkdownSection1.6` use to pin their own heading level, and what every other content class (`MarkdownParagraph`, `MarkdownListItem`, `MarkdownCodeBlock`, `MarkdownBlockQuote`) uses for its own token-type identity, not just headings. - `@alias(value=, type=)` (`alias.py`, `alias_type.py`'s `AliasType.LITERAL/SPACE_SEPARATED/REGEX`) attaches identity metadata used only for heading-*text* matching at parse time (`alias_match.match_alias`), independent of `@markdown`, and never used for rendering.

A concrete section type subclasses the appropriate `MarkdownSectionN` for its depth (e.g. `CharacteristicInformation(MarkdownSection2)` for `## Characteristic Information`) and, separately, may declare `@alias(...)`. Alias matching is a literal, case-sensitive string comparison against the heading's raw inline source text: there is no plain-text extraction step and no formatting-stripping. `## **Extensions**`'s heading text is the literal string `"**Extensions**"`, which matches only an alias declared as `"**Extensions**"`, never an unformatted `"Extensions"` -- formatting markup is part of the compared value, not ignored. When a class declares no `@alias` at all, the default is `AliasType.SPACE_SEPARATED`'s own derivation -- `heading_text == space_separated_name(cls.__name__)`, e.g. `RelatedInformation` matches a `"Related Information"` heading -- equivalent to an implicit `@alias(value=<space-separated class name>, type=AliasType.SPACE_SEPARATED)` (corrected in v1.4.0 below; v1.2.0/v1.3.0/v1.3.1 wrongly specified this default as a literal match against `cls.__name__` verbatim, and all of v1.0.0-v1.4.0 wrote the still-hypothetical decorator as `@some_annotation(alias=...)` rather than the two decorators, `@markdown/@alias(value=...)`, actually implemented -- see v1.5.0 below). `@alias` remains opt-in for *customizing* the comparison away from that default (an explicit literal value with different wording/casing/suffixes/formatting, or a regex), not for enabling matching in the first place: an undecorated `MarkdownSection` subclass is always checked against something. A class whose heading text is data rather than a fixed schema label (e.g. a document's own H1 title) should declare an explicit `@alias(value=".+", type=AliasType.REGEX)` to accept any non-empty heading text (v1.3.1) -- there is no separate opt-out of alias matching for this case; the `SPACE_SEPARATED` default alone would still pin such a title to a fixed, class-name-derived value.

Consequences

Future document types (req, uc, ac) can define their schema declaratively and reuse the same recursive parse/render/validate engine instead of hand-rolling a parser per type. Nested sections (e.g. a use case's `Characteristic Information` sub-fields) become real typed, individually-validated fields rather than opaque validated blobs. Rendering is uniform and simpler than originally decided: every leaf class stores its own rendered, `mdformat`-normalized extent verbatim in a private `_value: str` (not replayed `_tokens`, as first sketched here -- see v1.5.0 below), so inline formatting inside a heading round-trips for free instead of requiring a dedicated fidelity mechanism. Each `MarkdownSectionN` base

class gives every section type a structural invariant "for free" via `MarkdownSection.get_extent`'s own stop condition (no illegally-nested same-or-higher-level heading), without hand-writing a dedicated `model_validator` per model -- though the dedicated per-class validator originally envisioned for this was drafted and left commented out/inert (see v1.5.0 below), so today this invariant is upheld structurally by `get_extent`/the trailing-completeness check alone, not doubly enforced. `alias` is now purely parse-time identity metadata with no rendering role, cleanly separating "what this section is called" from "what it looks like when rendered." The recursive, extent-based parsing algorithm is robust against silently misassigning an absent optional field's slot to the wrong content (unlike pure positional matching), but this safety depends on implementing the trailing completeness check at every nesting level -- omitting it would silently reintroduce the same class of bug for out-of-declared-order or unrecognized sections. Every schema-fixed section requires its own small dedicated class (e.g. `CharacteristicInformation`, and eventually `MainSuccessScenario`, `Extensions`, etc.), trading a larger number of small classes for self-documenting, independently-constrainable types, consistent with how `models/adr/v1` is already structured. Adds new base-library dependencies (a markdown tokenizer such as `markdown-it-py`, plus YAML frontmatter parsing) to the library's dependency set, since parsing is core behavior, not CLI/MCP-only. The existing ADR pipeline is deliberately left on its own hand-rolled implementation for now; unifying it onto this generic engine, if ever done, is a separate future decision and out of scope here.

Literal, formatting-sensitive alias matching (v1.2.0) means a heading whose formatting differs from a declared alias's formatting no longer matches (e.g. `## **Extensions**` vs. alias `"Extensions"`) -- authors must declare the alias with the exact formatting the heading actually uses, or drop the formatting from the heading; this part of v1.2.0 is unaffected by the v1.4.0 correction below.

Defaulting an undecorated class to `AliasType.SPACE_SEPARATED`'s derivation of its own `cls.__name__` (corrected in v1.4.0; v1.2.0/v1.3.0/v1.3.1 wrongly specified this default as a *literal* match against `cls.__name__` verbatim) is still an opt-out-by-exception default rather than the original "no alias means accept anything" enforcement -- every section class is always checked against something -- but it is a far less disruptive default than the erroneous literal one: a multi-word-named section class whose heading is that class name's natural space-separated form (e.g. `RelatedInformation` matching a "Related Information" heading) now matches automatically, with no `@alias` needed at all. An explicit `@alias` is only required when the heading differs from that derivation -- different wording/casing, a "(required)"/"(optional)" suffix, inline formatting markup, or a hyphen the derivation doesn't produce (e.g. `SubVariations` vs. `"Sub-Variations"`) -- exactly the case already covered by `test_uc_example.py`'s fixtures. Any already-existing fixture/model class whose explicit `@alias(type=AliasType.SPACE_SEPARATED)` (or an equivalent literal value) merely re-states what this corrected default now does automatically carries a now-superfluous annotation that should be removed (e.g. `various_models.py`'s `CharacteristicInformation`, `test_markdown_section_levels.py`'s `TopLevel..SixthLevel`). This resolves v1.2.0's open follow-up item (1) (reconciling `models/md`'s existing fixtures against the default) differently than originally anticipated: instead of adding aliases to every multi-word-named class, the default itself was wrong and is now fixed, so most such fixtures need no annotation at all.

A document's own H1 "title" leaf type (data, not a fixed schema label) resolves the above (v1.3.0, refined v1.3.1) via the already-decided regex alias mode: `@alias(value="."+", type=AliasType.REGEX)` accepts any non-empty single-line heading text, since `re.fullmatch("."+", heading_text)` succeeds for any string with at least one character (short of an embedded newline) and fails on an empty string. No new opt-out-of-alias-enforcement concept is needed or introduced; "accept anything non-empty" is just an unusually permissive alias, matched the same way every other alias is. This still requires such a leaf type to be decorated explicitly -- "no alias declared"

defaults to the SPACE_SEPARATED derivation of the class's own name (see above), not "accept anything" -- so a class that wants that behavior must say so via `+.+`, not by omission. `+.+` rather than `.*` is a deliberate rejection of an empty title -- a document is not considered validly titled if its H1 heading has no text.

Confirmation

Verify `models/md/` has no import dependency on `models/adr/v1/`. Verify `MarkdownSection.get_extent` stops at any sibling/ancestor heading (level $\leq$ its own) while folding a nested *deeper* heading into the current section's extent, so a valid instance never fails against its own heading -- this is the actual mechanism providing the structural invariant originally assigned to a per-class `model_validator`; that validator (`markdown_section.py`'s `validate_heading_structure`, `markdown_section1.py..6.py`'s `validate_headings`) exists but is currently commented out/inert, a known follow-up, not a passing check. Verify `str().__repr__` round-trips inline formatting inside a heading (e.g. `## **Extensions**`) exactly, via the stored, `mdformat-normalized_value: str` (not replayed tokens, as originally sketched here). Verify alias matching is a literal, case-sensitive string comparison against the heading's raw inline source text, with no plain-text extraction and no formatting-stripping: a formatted heading (`## **Extensions**`) does NOT match an alias declared as the unformatted text ("`Extensions`"); it matches only the literal formatted string ("`**Extensions**`"). Verify a class with no `@alias` declared at all still enforces a match -- via `AliasType.SPACE_SEPARATED`'s derivation of `cls.__name__` (`heading_text == space_separated_name(cls.__name__)`), not a literal match against the raw class name, and not accepting any heading text. Verify a document's own H1 title type (data, not a fixed schema label) accepts any non-empty heading text by declaring a regex alias `+.+ (@alias(value="+.+", type=AliasType.REGEX))`, not by any special-cased exemption from alias matching -- `re.fullmatch("+.+", heading_text)` matches any non-empty single-line heading text, including one containing inline formatting markup, but rejects an empty heading text. Verify the recursive parser assigns `None` (never a mis-bound value) when an optional field is absent from the document rather than binding a later field's content to it (see `MarkdownStr.process_field`'s optional handling). Verify a trailing completeness check (`remaining_text == ""`) raises a parse error on any unconsumed/out-of-order heading rather than silently ignoring it, at every nesting level. Verify a missing required field raises a clear parse error. Verify a fixture model reproducing `tests/feat-5-md-model-parser/uc_example.md`'s full nested structure (Characteristic Information's `###` children included) round-trips through `MarkdownSection1.from_text/str()` -- not `parse_document/render_document`, names that were never implemented. Verify no opt-in `RoundTrip()` marker exists to apply in the first place: byte-exact round-trip is the engine's unconditional default behavior, not an opt-in feature to gate (see `.specmgr/feat/feat-5-md-model-parser/README.md`'s REQ-004/REQ-005).

More Information

Revised 2026-08-11 (v1.5.0): reconciled this ADR's Decision Outcome/Consequences/Confirmation against the actual `models/md/` implementation, per repo-owner request while reviewing `.specmgr/feat/feat-5-md-model-parser/README.md`. Corrected: (1) package path `models/markdown/v1/` -> `models/md/` (no `v1/` subfolder); (2) `MarkdownStr` retaining parsed tokens (`_tokens`) -> a private `_value: str` holding rendered, `mdformat-normalized` text instead, decided in the README's own REQ-002 but never back-ported here; (3) `MarkdownHeading1..MarkdownHeading6` -> the actually-shipped `MarkdownSection/MarkdownSection1..MarkdownSection6`; (4) the placeholder `@some_annotation(alias="...")` annotation, explicitly flagged in v1.0.0 as "to be formalized... during

implementation" -> formalized as **two** independent class-level decorators, `@markdown(type=, tag=)` (structural type/tag identity, also used by non-heading classes like `MarkdownParagraph/MarkdownListItem`) and `@alias(value=, type=)` (heading-text identity only), not the single decorator originally sketched; (5) the per-class `model_validator` asserting no same-or-higher nested heading -> noted as drafted but currently commented out/inert (`markdown_section.py`'s `validate_heading_structure`, `markdown_section1.py`..6.py's `validate_headings`), with the equivalent protection actually delivered by `MarkdownSection.get_extent`'s own stop condition plus `from_text`'s trailing-completeness check; (6) Confirmation's `parse_document/render_document` API and `tests/feat-3-md-str-constraints/uc_example.md` fixture path, neither ever implemented/correct -> `MarkdownSection1.from_text/str()` and `tests/feat-5-md-model-parser/uc_example.md`; (7) Confirmation's `RoundTrip()` bullet, moot twice over (byte-exact round-trip is now the unconditional default, and the generic constraint-marker framework that would have hosted such an opt-in marker was itself dropped as speculative -- see the README's REQ-005 rescoping, same date). No behavior changed by this revision; it is documentation-only, correcting drift that accumulated silently across the class-hierarchy pivot (v1.1.0) and the alias-matching corrections (v1.2.0-v1.4.0) without ever being reflected back into Decision Outcome/Consequences/Confirmation. Tracked in `.specmgr/feat/feat-5-md-model-parser/README.md` (GitHub issue #5).

Revised 2026-08-11 (v1.4.0): corrected an error introduced in v1.2.0 and repeated unchanged through v1.3.0/v1.3.1: the default for a class declaring no `@alias` at all was wrongly specified (and, until now, wrongly implemented in `alias_match.py`) as a *literal* match against `cls.__name__` verbatim. Per explicit repo-owner correction, it must instead be `AliasType.SPACE_SEPARATED`'s own derivation -- `heading_text == space_separated_name(cls.__name__)` -- exactly the behavior the `@alias` decorator itself already uses as *its own* default (type: `AliasType = AliasType.SPACE_SEPARATED` in `alias.py`) when applied with no explicit `type=`; the no-decorator-at-all case now simply matches that same decorator default instead of silently diverging from it into `LITERAL`. Concretely this means e.g. `RelatedInformation` matches a "Related Information" heading with no `@alias` needed at all, whereas v1.2.0's text (and, until this revision, `match_alias`'s actual code) required an explicit alias for that case. `AliasType.LITERAL` and `AliasType.REGEX` remain unaffected and still require an explicit, opt-in `@alias` exactly as before -- only the *implicit, no-decorator* default changes. This also resolves v1.2.0's open follow-up item (1) (reconciling `models/md/`'s existing fixtures against the no-alias default) more simply than anticipated there: rather than adding an explicit alias to every multi-word-named class, the wrong default is fixed, so most such classes need no annotation at all. Updated: `alias_match.py`'s `match_alias` (no-metadata branch) and its docstring; `markdown_section.py`'s `from_text` docstring; `alias_type.py`'s stale `LITERAL-described-as-"the default alias type"` docstring line (already inconsistent with `alias.py`'s actual `SPACE_SEPARATED` decorator default, independent of this fix, corrected alongside it). Removed now-superfluous explicit annotations that merely restated the corrected default: `various_models.py`'s `CharacteristicInformation (@alias(value="Characteristic Information", type=AliasType.LITERAL))`, and `test_markdown_section_levels.py`'s `TopLevel..SixthLevel (@alias(type=AliasType.SPACE_SEPARATED))`, six classes). No fixture needed a newly-added annotation: `various_models.py`'s `RelatedInformation` was already correctly left undecorated and now matches under the corrected default -- this also fixes the two tests that were failing under the previous (incorrect) code, `test_markdown_section.TestMarkdownSectionStr.test_composite_document_reemits_every_heading_and_body` and `test_markdown_str.TestFromText.test_main_document_from_text`. Updated `test_alias_match.py/test_markdown_section.py`'s no-`@alias`-default tests (previously asserting the literal-class-name behavior) to assert the corrected `SPACE_SEPARATED` behavior instead. Tracked in `.specmgr/feat/feat-5-md-model-parser/README.md` (GitHub issue #5).

Revised 2026-08-11 (v1.3.1): changed the accept-any-heading regex for a document's H1 "title" leaf type from `.*` (zero or more characters) to `.+` (one or more characters), per explicit repo-owner direction: an empty heading must not be accepted as a valid document title. `re.fullmatch(".+", heading_text)` now rejects an empty `heading_text`; it still matches any other single-line text, including inline-formatted text. Updated throughout Decision Outcome/Confirmation/Consequences (previously written as `.*` in v1.3.0, which would have accepted an empty title).

Revised 2026-08-11 (v1.3.0): resolved v1.2.0's open item (2) -- the missing opt-out-of-alias-enforcement mechanism for a document's own H1 "title" leaf type -- by reusing the already-decided regex alias mode instead of designing a new concept: `@some_annotation(alias=".+", type=REGEX)` (originally written as `.*`, corrected in v1.3.1 above) accepts any single-line heading text, verified empirically against Buy Goods, arbitrary text, and inline-formatted text. No new mechanism, marker, or exemption from alias matching was needed. Open item (1) (reconciling `models/md/`'s existing fixtures against the new literal/no-alias-defaults-to-class-name behavior) remains open, as does actually updating the code (`alias_match.py`) to match this ADR -- this revision is documentation-only, same as v1.2.0.

Revised 2026-08-11 (v1.2.0): changed alias matching from "extract plain text, ignore inline formatting" to a literal, case-sensitive comparison against the heading's raw inline source text (formatting markup included and significant), and changed the default for an undecorated class from "no alias declared means accept any heading text" to "no alias declared means a literal match against `cls.__name__`" (this second change was itself an error, corrected in v1.4.0 above). Prompted by re-reading `tests/feat-5-md-model-parser/req_parser.py`'s design notes against the actual shipped `models/md/` implementation (`alias_match.py`), which already compared heading text via raw `.content` (not a plain-text-children walk) and already treated "no `@alias`" as "accept anything" -- both diverging silently from this ADR's original v1.1.0 text without ever being recorded as a decision. This revision formalizes those two behaviors as the intended design going forward (repo-owner direction: "LITERAL means LITERAL", and undecorated classes should default to a match rather than no check at all), rather than treating them as undocumented drift. Known open follow-up (not resolved by this revision, see Consequences): (1) reconciling `models/md/`'s existing fixtures (`various_models.py`, `test_uc_example.py`) that rely on the old "no alias = accept anything" default for multi-word-named classes -- resolved in v1.4.0 above, differently than anticipated here; (2) designing the still-missing explicit opt-out-of-alias-enforcement mechanism for a document's own H1 "title" leaf type, which can no longer rely on "no alias declared" now that this default is a match against something -- resolved in v1.3.0/v1.3.1 above. Tracked in `.specmgr/feat/feat-5-md-model-parser/README.md` (GitHub issue #5).

Revised 2026-08-08 (v1.1.0): superseded the original `Annotated[Heading(tag=, alias=)]` field-metadata mechanism with the `MarkdownHeading1..MarkdownHeading6` class hierarchy + class-level alias + sequential cursor-based recursive-descent parser described above; see `tests/feat-5-md-model-parser/req_parser.py` (top-of-file notes block) for the up-to-date sketch this ADR reflects, including remaining open items not yet finalized. Originating sketch and fixture: `tests/feat-5-md-model-parser/req_parser.py`, `tests/feat-3-md-str-constraints/uc_example.md` (the fixture file is reused as-is; this decision does not otherwise affect the `feat-3-md-str-constraints` feature, which is a separate, regex-based string-constraint type tracked independently). Related: ADR 4c6119c9-532f-4629-8977-108e78304f48 (ADR-specific parse-validate-render pipeline, not superseded by this decision). Tracked in `.specmgr/feat/feat-5-md-model-parser/README.md` (GitHub issue #5).

```
status: accepted
date: '2026-08-27'
decision-makers: OpenCode agent + user decision
id: 36905d5b-8057-4294-8665-c7eed5534db0
version: 1.0.0
```

Consolidate whole-body update and status-change tools into generic type-dispatched tools

Context and Problem Statement

The specmgr MCP server currently exposes 15 near-duplicate mutation tools for what are two conceptual operations: seven per-domain whole-body updates (`update_req`, `update_uc`, `update_tsk`, `update_qa`, `update_prb`, `update_gol`, `update_rsk`), seven per-domain status changes (`set_status_req`, `set_status_uc`, `set_status_tsk`, `set_status_qa`, `set_status_prb`, `set_status_gol`, `set_status_rsk`), and ADR's own `set_status`. Each tool shares the same shape — id resolution in one domain directory, validation, frontmatter carry-over, updated bump, write — and differs only in domain vocabulary. LLM/agent clients see 15 entries in the MCP tool list for 2 conceptual operations, and every future document domain (e.g. the planned `ac`) would add more of the same duplicates, growing the surface linearly with the number of domains.

Decision Drivers

- A simpler tool surface: the two conceptual operations should be exposed as two tools, not fifteen near-duplicates.
- Id resolution must not require an all-domains directory scan on the write path, and must not introduce per-domain v4-UUID-collision ambiguity — uuid-only id resolution was considered and rejected (per-domain v4 UUIDs are not guaranteed unique across domains).
- The calling client already knows the domain it is operating on (the same vocabulary as the frontmatter `type` field), so passing it explicitly costs the client nothing.
- Preserve the existing invariants: the filesystem is the sole source of truth (ADR 33c5ab08-ff58-4c73-8c32-23abaf3838e3) and validation happens before any write (nothing is written on validation failure).

Considered Options

- Option 1: two generic tools in `general/tools/` with an explicit type parameter — `update(id, type, content, begin, end)` covering the seven whole-body domains and `set_status(id, type, status, superseded_by)` covering all eight domains including `adr` — each dispatching to a private, verbatim-porting per-domain adapter. Chosen.
- Option 2: generic tools that resolve the id by uuid alone, scanning every domain directory to locate the matching document.

- Option 3: keep the 15 per-domain tools unchanged.

Decision Outcome

Option 1: two generic, type-dispatched tools — `update(id, type, content, begin, end)` in `general/tools/update.py` for the seven whole-body domains (`req`, `uc`, `tsk`, `qa`, `prb`, `gol`, `rsk`), and `set_status(id, type, status, superseded_by)` in `general/tools/set_status.py` for all eight domains including `adr`. The explicit type parameter keeps id resolution single-domain (no directory scan, no cross-domain UUID ambiguity), matches the domain vocabulary the calling client already has, and reduces the tool surface from 15 near-duplicate entries to 2. Each domain's semantics are preserved 1:1 by a private adapter that is a verbatim port of the deleted tool body, so the filesystem-is-source-of-truth and validate-before-write invariants are untouched.

Consequences

- Bad (breaking): the 14 per-domain tools are removed outright, and ADR `set_status`'s signature gains a required type parameter — existing ADR callers must now pass `type="adr"`. The package is 0.x and the MCP tool list is the only client contract; the breaking change is recorded in `CHANGELOG.md`.
- ADR is excluded from `update` — its section-level MADR contract (`update_frontmatter/update_section/option_*`, ADR 71fd95d7-07f2-466f-81aa-d29b7e3ef34c) has no whole-body replace by design — but is included in `set_status` with the `superseded_by` special case: `superseded_by` composes the status as "superseded by {superseded_by}", and `superseded_by` given with any type other than "adr" raises `ValueError` before any file access.
- The `update` line-range contract: optional 1-based, inclusive body-line coordinates `begin/end`, with `N+1` as the EOF sentinel (`begin = end = N+1` appends at end of body; `end = N+1` extends the range through the last line). The spliced result is validated as a whole document before anything is written (splice-then-validate-whole), and the YAML frontmatter is never addressable (coordinates are body-relative by construction).
- Line numbers for range updates are served by a new `get_<d>(raw=True)` parameter returning the frontmatter-stripped body text verbatim — tool-first per ADR ddfb1109-422d-4507-8dbc-dc5e4bec9614 (agents invoke tools more reliably than parameterized resources); re-introducing `specmgr://<d>/{id}` resources was considered and rejected.
- Good: future domains (e.g. `ac`) add one dispatch entry per generic tool (plus a `raw` getter parameter), not new tools.

Pros and Cons of the Options

Option 1: Generic tools with an explicit type parameter

Pros

- Minimal tool surface: two tools for the two conceptual operations (the feature ends at 71 tools / 25 resources / 19 prompts, from 84/25/19: -15 +2), instead of 15 near-duplicate entries in the MCP tool list.

- The calling client already knows the domain — it is the same vocabulary as the frontmatter type field — so the explicit parameter costs the client nothing, and id resolution stays single-domain: no all-directories scan on the write path, no per-domain v4-UUID-collision ambiguity.
- Every future domain (e.g. the planned ac) adds one dispatch entry per generic tool, keeping the surface flat as domains grow.
- Preserves all per-domain semantics: each adapter is a verbatim port of the deleted tool body (same lock, same load_by_id, same frontmatter carry-over and updated bump, same write path, same domain not-found error), and the filesystem-is-source-of-truth and validate-before-write invariants are untouched.

Cons

- Breaking change for 0.x clients: the 14 per-domain tools disappear, and ADR set_status's signature gains a required type (existing ADR callers must now pass type="adr").
- ADR needs special-casing in set_status (the superseded_by composition) and is excluded from update by design (its MADR section-level contract has no whole-body replace).

Option 2: uuid-only id resolution scanning all domain directories

Pros

- Shortest client call: no type parameter; any document in any domain is addressable by id alone.

Cons

- Full-directory scan on every write: all domain directories must be traversed and every file parsed to locate the matching id, and the cost grows with each added domain on the write path.
- Per-domain v4 UUIDs are not guaranteed unique across domains, so a collision between two domains makes the id ambiguous — the server would have to pick one arbitrarily or raise a new class of errors.
- Loses the explicit domain vocabulary clients already use everywhere else (the frontmatter type field) and obscures which domain's semantics (status vocabulary, lock, write path) are actually being applied.

Option 3: Keep the per-domain tools

Pros

- No breaking change; existing clients keep working unchanged.
- No dispatch-table machinery; each tool remains a simple single-domain wrapper.

Cons

- The MCP tool list carries 15 near-duplicate entries for 2 conceptual operations, inflating every client's tool context.
- Every future domain adds more near-duplicate tools (a update_<d> / set_status_<d> pair per domain), growing the surface linearly with the number of domains.

- LLM clients must pick among the duplicates for each operation, which risks mis-selection and makes the surface harder to document, test, and keep consistent.

More Information

- Feature plan and progress: `.specmgr/feat/feat-22-consolidate-mutation-tools/README.md`.
- Related ADRs: `ddfb1109-422d-4507-8dbc-dc5e4bec9614` (id-based document reads are tools, not resources), `71fd95d7-07f2-466f-81aa-d29b7e3ef34c` (the ADR `update_section` contract that update deliberately does not extend to ADR), `33c5ab08-ff58-4c73-8c32-23abaf3838e3` (filesystem is the sole source of truth), `ece4554b-725c-4f76-bc04-5d2b760363d2` (domain-first hierarchy — the generic tools live in the cross-cutting `general/` package).

ADR `ec9f5262-9912-49d0-903f-fcfb54f28c13`

```
status: accepted
date: '2026-08-19'
decision-makers: OpenCode agent + user decision
id: ec9f5262-9912-49d0-903f-fcfb54f28c13
version: 1.0.0
```

Expose *list as paged MCP tools (list)*, not resources

Context and Problem Statement

The five `<domain>_list` MCP resources (`specmgr://adr/list`, `specmgr://req/list`, `specmgr://uc/list`, `specmgr://tsk/list`, `specmgr://qa/list`) each did a full, unbounded directory scan and returned a bare `list[<D>Summary]` on every call. As the number of documents in a base directory grows, this becomes increasingly expensive and eventually unwieldy for a calling agent to consume in one shot. Pagination (`max_results/offset`) was raised as `feat-7-various-improvements Task 0.15/REQ-002`, but MCP resources can only be parameterized via URI-template path segments, not arbitrary query parameters -- so `max_results/offset` cannot be added to a `@mcp.resource()` without contorting the URI shape. This mirrors the reasoning already recorded in ADR `ddfb1109-422d-4507-8dbc-dc5e4bec9614` for `get_req`: resources are a poor fit whenever a read needs caller-supplied parameters.

Decision Drivers

- Pagination parameters (`max_results/offset`) do not fit MCP resources, which are URI-template-only.
- Consistency with ADR `ddfb1109-422d-4507-8dbc-dc5e4bec9614`'s precedent: prefer tools over resources for parameterized, on-demand reads.

- Reuse an existing, proven paged-result shape rather than invent a new one, for consistency across this project's own MCP servers.
- Preserve the exact current scan/sort/skip-broken-file semantics -- no behavioral regression for callers relying on today's full listing.
- Keep the five domains' summary models on one shared, documented base field set where the dependency graph allows it.

Considered Options

- Option 1: Keep `<domain>_list` as resources, encode `max_results/offset` into the URI template (e.g. `specmgr://req/list/{offset}/{max_results}`)
- Option 2: Convert all five `<domain>_list` resources into `@mcp.tool() list_<domain>` tools accepting `max_results/offset`, returning a shared `PagedResult[T]` wrapper
- Option 3: Keep resources unbounded as-is and defer pagination indefinitely

Decision Outcome

Chosen option: "Option 2: Convert all five `<domain>_list` resources into `list_<domain>` tools returning `PagedResult[T]`", because it is the only option that gives callers real, bounded pagination without abusing the URI-template mechanism, and it follows the precedent already accepted in ADR [ddfb1109-422d-4507-8dbc-dc5e4bec9614](#). Each `list_<domain>` tool accepts `max_results/offset` (default page size 25, cap 100; out-of-range inputs are clamped, not errored), and returns a `PagedResult` shape -- `total`, `offset`, `max_results`, `truncated`, `results` -- taken verbatim from this project's own `asdstel00` MCP server (`word_list`, `rules_examples`), rather than inventing a new contract. Each tool still fully materializes its domain's summary list first (identical scan/sort/skip-broken-file behavior as the retired resource) and then slices it in memory, so `total` reflects only parseable documents and no existing behavior regresses.

Four of the five domains' `*Summary` models (`ReqSummary`, `UcSummary`, `TskSummary`, `QaSummary`) now subclass a new shared `general/models/summary.py::DocSummary` base (`id`, `title`, `status`, `ref`). `AdrSummary` (`models/adr/v1/summary.py`) is a deliberate, permanent-for-now outlier: it stays field-identical to `DocSummary` but does not subclass it, because `models/adr` is a dependency-free base-library module (no `mcp` import, per `AGENTS.md`'s "models location" note), while `general/models` transitively requires the `mcp` extra through `general/__init__.py`'s unconditional import of `general.tools/general.resources/general.prompts`. Making `AdrSummary` subclass `DocSummary` would silently add a new `mcp` dependency to the base library. This is accepted as-is, with a known future redesign path: ADR is the only domain not yet using the generic markdown parser, and a future ADR-domain redesign is expected to revisit this asymmetry; a structural-equivalence test (`tests/general/models/test_summary.py`) keeps the two field sets in sync in the meantime.

This work was split out of `feat-7-various-improvements` Task 0.15 into its own feature folder, `feat-13-list-paging` (GitHub issue #13), and closes `feat-7's REQ-002/ACC-002` (the pagination decision). It also advances, but does not fully close, `feat-7's REQ-001/ACC-001` (the shared list-output contract): the contract is now shared and documented across all five domains, but ADR's summary shares it structurally rather than via inheritance, per the outlier above.

Consequences

Good, because all five `list_<domain>` tools are now reliably invocable by agents with real pagination, matching the tool-first precedent already set for `get_req`. Good, because the paged-result shape is proven and reused (from `asdste100`), not invented, keeping this project's own MCP servers consistent with each other. Good, because scan/sort/skip-broken-file behavior is unchanged -- total still reflects only parseable documents, and each domain's own parse-failure exception tuple (e.g. ADR's (`AdrParseError`, `ValidationError`) vs. the other four's (`AssertionError`, `ValidationError`)) is preserved exactly. Bad, because the five `<domain>_list` MCP resources are gone; any external client that was reading them as context-attachment resources (per the original rationale in ADR 7531106b-074b-4bd8-a83a-e433d01676e2) must switch to calling the `list_<domain>` tool instead. Bad, because `AdrSummary` remains a visible, permanent-for-now asymmetry against the shared `DocSummary` base -- readers must know this is intentional (dependency-graph-driven), not an oversight, and that a future ADR-domain redesign may eventually resolve it. Neutral, because this partially reverses the listing side of ADR 7531106b-074b-4bd8-a83a-e433d01676e2 (which added `specmgr://adr/list` as a resource) while leaving that ADR's by-id resource decision (`specmgr://adr/{id}`) untouched.

More Information

Supersedes the listing-resource half of ADR 7531106b-074b-4bd8-a83a-e433d01676e2 ("Expose listing and by-id reads as MCP resources in addition to tools") for all five domains; that ADR's by-id resource decision is unaffected. Extends the tool-over-resource precedent of ADR `ddfb1109-422d-4507-8dbc-dc5e4bec9614` (`get_req`) to the listing case. Implemented in `.specmgr/feat/feat-13-list-paging/README.md` (split out of `feat-7-various-improvements` Task 0.15), which tracks the full per-domain task breakdown; see that feature folder's Decisions Made log for implementation-level detail not repeated here.

ADR `ddfb1109-422d-4507-8dbc-dc5e4bec9614`

```
status: accepted
date: '2026-08-15'
decision-makers: OpenCode agent + user decision
id: ddfb1109-422d-4507-8dbc-dc5e4bec9614
version: 1.0.0
```

Expose id-based REQ document reads as a tool (`get_req`), not a resource

Context and Problem Statement

`feat-6-requirement-artifact` Task 3.17 deliberately exposed id-based single-document reads for the REQ domain only as an MCP resource (`specmgr://req/{id}`, `req/resources/req_get.py`), explicitly superseding an earlier-considered `get_req` tool -- the stated rationale was "id-based single-document

read is a resource only, everything else in this surface is a tool". This mirrored the *shape* of the ADR domain's `specmgr://adr/{id}` resource, except ADR also ships a `get_adr` tool alongside it, so the two domains were never actually symmetric.

In practice, LLM/agent clients calling this MCP server fail to reliably invoke `specmgr://req/{id}` to retrieve a requirement by id -- resources are a much less-used affordance for on-demand, parameterized lookups than tools are, in current agent tool-use patterns. This defeats the purpose of exposing the lookup at all: the calling model either skips the read entirely, or falls back to less reliable paths (e.g. reading `specmgr://req/list` and guessing, or trying to read the underlying `.md` file directly off disk). This was raised as feat-7-various-improvements Task 0.9, which also asks whether the same reasoning should retroactively apply to the already-shipped `get_adr` tool / `specmgr://adr/{id}` resource pair.

Decision Drivers

- Reliability: in observed practice, LLM agents invoke tools far more reliably than resources for on-demand, parameterized (id-based) data retrieval.
- Consistency across document domains (`adr`, `req`, and future `uc/ac`) for the same conceptual operation ("read one document by id").
- Avoid permanently maintaining two parallel code paths (tool and resource) for the same read when the resource path is rarely, if ever, actually exercised by callers.
- Minimize churn to the already-shipped, working ADR domain surface, which has no reported reliability problem to date.

Considered Options

- Option 1: Add `get_req` tool, keep `specmgr://req/{id}` resource (tool and resource coexist, mirroring how ADR already works)
- Option 2: Add `get_req` tool, remove `specmgr://req/{id}` resource entirely (REQ becomes tool-only for id-based reads)
- Option 3: Do nothing -- keep REQ resource-only, reaffirming feat-6 Task 3.17's original decision

Decision Outcome

Chosen option: "Option 2: Add `get_req` tool, remove `specmgr://req/{id}` resource entirely", because the resource has demonstrated the exact reliability problem this ADR describes, and keeping a rarely-invoked resource around after adding the tool would only add maintenance surface (two code paths, two test suites, two docstring entries) for a path that isn't actually helping callers.

This ADR explicitly does **not** extend the change to the ADR domain: `specmgr://adr/{id}` (`adr_get`) stays coexisting with the already-shipped `get_adr` tool, unchanged. This is a deliberate, accepted cross-domain divergence -- not an oversight -- recorded here so a future reader does not assume the two domains were meant to be symmetric. Any future document domain (`uc/get_uc`, `ac/get_ac`, ...) should follow the newer REQ precedent (tool-only for id-based reads) rather than the older ADR precedent, unless a specific reason to add a resource counterpart is identified at that time.

Consequences

Good, because REQ's id-based read becomes reliably invocable by agents via a normal tool call, matching every other REQ lifecycle operation (`create_req`, `update_req`, `set_status_req`, `validate_req`), which are already tools. Good, because it establishes a clear default for future document domains: id-based single-document read is a tool first; a resource counterpart is only added if a concrete need for non-tool-mediated context retrieval emerges. Bad, because the ADR and REQ domains now visibly diverge in their tool/resource surface for the same conceptual operation (ADR keeps both `get_adr` tool and `adr_get` resource; REQ has only `get_req`) -- readers of both domains side by side must know this is intentional, not a bug, which is why it is written down here. Neutral, because this reverses part of feat-6 Task 3.17's original design without reopening feat-6 itself -- feat-6's README is annotated with a pointer to this ADR rather than rewritten.

More Information

Supersedes the `specmgr://req/{id}`-only decision recorded in `feat-6-requirement-artifact/README.md` Task 3.17. Tracked as `feat-7-various-improvements` Task 0.9 (sub-tasks 0.9.1-0.9.13).

ADR e369ee2e-3353-4f92-991c-6367d76d832e

```
status: accepted
date: '2026-08-05'
decision-makers: dfch
id: e369ee2e-3353-4f92-991c-6367d76d832e
version: 1.0.0
```

Organize development artifacts in `.specmgr` with feature-driven work units

Context and Problem Statement

The project maintains two documentation folders with an unclear split: `docs/` — published, generated documentation (API docs, ADRs, specifications), which reflects the current state of the project — and `doc/` — development progress notes, planning artifacts, research, which has no clear ongoing purpose once this ADR's structure exists. As part of adopting this ADR's outcome, `doc/` is dissolved: its content is migrated (manually, see Consequences) into the new structure below, and the `doc/` folder is retired. Development artifacts (plans, progress tracking, work-unit status) need a structured, agent-friendly location that is: (1) separate from published documentation (`docs/`); (2) organized by feature/work-unit for easy agent reference; (3) generic enough to serve as a template for future projects using `specmgr` as a toolkit.

Decision Drivers

- Agent-friendly reference paths: agents should reference specific feature paths inline (e.g., "See `.specmgr/feat/feat-001-adr-toc/README.md`"), keeping agent instructions lean and focused
- Toolkit reusability: structure should be generic enough for future projects adopting `specmgr` as a toolkit — the folder name itself (`.specmgr/`) is chosen for this reason, following the convention of tool-named dotfolders like `.github/`, `.vscode/`, `.docker/`
- Clear separation of concerns: development artifacts must be distinct from published documentation
- Version control and auditability: development progress should be tracked in git with full history

Considered Options

- Single `README.md` per feature containing both plan and progress
- Separate `README.md` (plan) and `progress.md` (status) per feature

Decision Outcome

Chosen option: "Option 1: `.specmgr` structure" — a single `README.md` per feature combining plan and progress, with an optional sibling `history.md` for rotating out older Recent Updates entries. Every feature `README.md` also carries a minimal YAML frontmatter block (`id`, `version`, `status`, `created`, `updated` — see that option's "Frontmatter" note for details). There is no separate GitHub Issue field or body line: the issue number is the NNN infix already embedded in `id` (the folder name, `feat-NNN-slug`) itself.

This is preferred over Option 2 (separate `README.md/progress.md`) for its simplicity: one file per feature, no cross-file cross-referencing needed to see the full feature story, and the single canonical Task List (status inline per task) removes the Implementation Plan/Execution Plan duplication that Option 2 still carries. See "Pros and Cons of the Options" below for the full tradeoff analysis, and that option's "Open Questions" for points intentionally left open for later decisions.

Consequences

Positive: - Agents can reference specific feature paths inline, keeping instructions lean and focused - Clear separation: agents only read what's relevant to their task - Structure is reusable for future projects adopting `specmgr` as a toolkit - Development progress is version-controlled and auditable - The `.specmgr/` folder (and its `feat/` work units) is committed to git like any other tracked path in the repo — no `.gitignore` exclusion — so history and review apply to it the same way they do to docs/ and source code

Negative: - Adds another top-level folder to the repo structure - Requires discipline to keep progress sections updated (hand-maintained, not auto-generated) - Migrating doc/'s existing content (e.g. `doc/adr-tool-plan.md`, `doc/refactor-domain.md`) into the new structure is done manually, one file at a time, once this ADR is adopted — no automated migration tooling is planned

Numbering convention: - `feat-NNN-slug` — NNN is the GitHub issue number for feature work tied to an issue. There is no separate `github_issue` frontmatter field or body line: `id` (the folder name itself)

is the single source of truth for the issue number, read by parsing its NNN infix. - Work started without a GitHub issue yet uses feat-0-slug (issue number 0) until/unless an issue is later opened for it

ADR vs. feature-level "Decisions Made" log: A decision belongs in a full ADR (under docs/adr/) if it: (a) is architecture/structure-level and affects more than one feature or the repo as a whole, (b) would be relevant to someone joining the project later trying to understand why something is the way it is, or (c) reverses/supersedes a previous ADR. A decision belongs in the feature's own "Decisions Made" log instead if it: (a) is scoped entirely to that one feature's implementation details, (b) wouldn't need to be found by searching ADRs later, and (c) doesn't constrain future features. Tie-breaker: if in doubt, write the ADR — it is cheap to write and already indexed by adr-toc, so overuse is low-cost, while under-use risks losing a decision in a feature folder no one will grep later.

Confirmation

For now, confirmation that new feat-NNN-slug/README.md files follow the chosen structure/template is done manually via PR review. Automated enforcement (e.g. a specmgr feat-* validation tool mirroring validate_adr) is deferred to future work, consistent with the other deferred-tooling items noted in the chosen option's Open Questions.

Pros and Cons of the Options

Option 1: .specmgr structure

```
.specmgr/  
├── feat/  
│   ├── feat-NNN-slug/  
│   │   ├── README.md  
│   │   └── history.md  
│   └── (optional)  
│       └── (other dirs as needed)
```

Feature work units
One folder per GitHub issue
Feature plan + progress (mandatory)
Archived older "Recent Updates" entries

File purposes: - README.md — Single file containing both the feature plan (requirements, acceptance criteria, task list, scope, dependencies, design notes) and progress tracking (current state, blockers, decisions made during implementation, links to related ADRs or PRs) - history.md — Optional sibling file. Holds older Recent Updates entries once README.md grows too long; README.md keeps only recent entries and links back to this file for anything older.

Frontmatter: Every feature README.md carries a YAML frontmatter block, mandatory fields id (the feat-NNN-slug folder name itself, not a generated UUID — unlike ADR frontmatter's server-generated id), version (semver, starts at 1.0.0), status (planning | in-progress | review | done), and created/updated (YYYY-MM-DD, updated bumped on every substantive edit). There is no separate GitHub Issue field, in frontmatter or body: the issue number is the NNN infix already embedded in id (i.e. the folder name, feat-NNN-slug) — 0 means no issue yet — so it is derived by reading id, never duplicated as its own field.

Template: README.md

```
---  
id: feat-NNN-slug  
version: 1.0.0
```

```
status: planning
created: YYYY-MM-DD
updated: YYYY-MM-DD
---
```

```
# Feature: [Feature Title]
```

```
## Plan
```

```
### Overview
```

Brief description of what this feature does and why it matters.

```
### Requirements
```

- REQ-001: [Functional requirement]
- REQ-002: [Non-functional requirement]
- REQ-003: [Constraint or dependency]

```
### Acceptance Criteria
```

- [] ACC-001: Verifies REQ-001 – [testable condition]
- [] ACC-002: Verifies REQ-002 – [testable condition]
- [] ACC-003: Verifies REQ-003 – [testable condition]

```
### Scope
```

What is included in this feature:

- Item 1
- Item 2

What is explicitly out of scope:

- Item A
- Item B

```
### Dependencies
```

- Depends on: [other feat-NNN-slug, ADR id, or external]
- Blocks: [other feat-NNN-slug]

```
### Design Notes
```

Any architectural decisions, patterns, or design rationale relevant to this feature.

```
### Related ADRs
```

- [ADR id]: [Title]
- [ADR id]: [Title]

```
### Task List
```

Single, canonical breakdown of work phases and tasks. Status lives on the task itself – there is no separate "planned" vs. "executed" list to keep in sync; a task's line **is** its current status. Update it in place as work progresses (edit, don't duplicate).

```
#### Phase 1: [Phase name]
```

```
- [x] Task 1.1: [description] – depends on: none – status: done (2026-08-01)
- [ ] Task 1.2: [description] – depends on: Task 1.1 – status: in-progress, ETA 2026-08-10
- [ ] Task 1.3: [description] – depends on: Task 1.2 – status: blocked (see Blockers)
```

```
#### Phase 2: [Phase name]
```

```
- [ ] Task 2.1: [description] – depends on: Task 1.3 – status: not-started
- [ ] Task 2.2: [description] – depends on: Task 2.1 – status: not-started
```

Note: If a task's scope changes mid-flight, edit its description in place; rely on git history (`git log -p`` on this file) to recover what was originally planned, rather than keeping a second copy of the task around.

```
## Progress
```

```
### Current Status
```

As of [YYYY-MM-DD]: [Brief status summary]

```
### Blockers
```

```
- [ ] Blocker 1 – [description, impact, mitigation]
- [ ] Blocker 2 – [description, impact, mitigation]
```

(Remove this section if no blockers.)

```
### Recent Updates
```

If this section grows too long, move older entries to `history.md`` in this same folder and leave a pointer here, e.g.:
``See history.md for updates before YYYY-MM-DD.``

```
#### Update [YYYY-MM-DDTHH:mm:ssz] (newest)
```

```
- Completed: [what was done]
- Next: [what comes next]
- Notes: [any relevant context]
```

```
#### Update [YYYY-MM-DDTHH:mm:ssz] (oldest)
```

```
- Completed: [what was done]
- Next: [what comes next]
- Notes: [any relevant context]
```

```
### Decisions Made
```

```
- [YYYY-MM-DD]: [Decision] – [Rationale]
- [YYYY-MM-DD]: [Decision] – [Rationale]
```

```
### Related PRs / Commits
```

```
- [PR #NNN](link): [description]
- [Commit hash](link): [description]
```

Pros: - Single file to maintain - Simpler structure: one file per feature - Plan and progress are always together in one document - Easier to see the full feature story (what was planned vs. what happened) in one place - Requirements and acceptance criteria are co-located with clear traceability - Single

Task List: no separate Implementation/Execution Plan pair to keep in sync — status is a property of each task line, not a duplicated list, so there is nothing to drift - Auditability of "what was planned vs. what actually happened" comes from git history on this one file, not from a hand-maintained duplicate - Recent Updates growth is bounded by rotating older entries into an optional `history.md`, keeping `README.md` itself lean - Frontmatter `id/version/status/created/updated` gives each feature folder a compact, machine-readable header, mirroring the ADR frontmatter's `status` field for consistency across both document types - No GitHub Issue duplication: the issue number is already encoded in `id`'s NNN infix, so there is nothing to keep in sync between a frontmatter/body field and the folder name itself

Cons: - File grows over time as progress updates (Recent Updates, Decisions Made) accumulate, even with rotation available - Plan and progress are intermingled, making it harder to extract just the plan for reference - No clear separation between "contract" (what we committed to) and "journal" (what actually happened) — relies on git history to reconstruct the original plan instead of a preserved, separate copy - Still hand-maintained/free-text: nothing currently enforces that a task's status field, or the frontmatter `status/updated` fields, are kept in sync with reality, or that `history.md` rotation actually happens - Deriving the GitHub issue number from `id`'s NNN infix requires parsing the folder name rather than reading a dedicated field — acceptable since `feat-NNN-slug` is already a fixed, documented convention

Open Questions: - Archival/lifecycle rule for the file once `status: done` (stay in place / archive / prune) — intentionally left undecided here; treated as a separate future project decision, not a gap in this ADR. - Rotation strategy for Recent Updates: rotating older entries into `history.md` is documented here as an available option; the exact trigger (manual vs. a fixed entry-count rule) and mechanics are left to the user/agent maintaining the feature folder to decide at the time, not prescribed by this ADR. - Template location: **resolved** — the template now exists at `.specmgr/_template/v1/README.md`, matching the versioned path scheme originally proposed here. - Whether to add further frontmatter fields later (e.g. `decision_makers`, `related_adrs`, `tags`) is left open; the current five-field frontmatter (`id`, `version`, `status`, `created`, `updated`) is a deliberate, minimal starting point, not a ceiling. - Recommendation (not yet built, non-blocking): a dedicated MCP tool (analogous to this project's `update_section/option_update` for ADRs) that flips one task's status field, or the frontmatter `status`, atomically, instead of relying on an agent/human to locate and hand-edit the right line.

Option 2: `.specmgr` structure with separate `README.md` (plan) and `progress.md` (status)

```
.specmgr/
├── feat/                                # Feature work units
│   ├── feat-NNN-slug/                  # One folder per GitHub issue
│   │   ├── README.md                   # Feature plan (mandatory)
│   │   └── progress.md                  # Status tracking (mandatory)
└── (other dirs as needed)
```

File purposes: - `README.md` — Contains the complete feature plan: requirements, acceptance criteria, implementation plan, scope, dependencies, design notes, any pre-implementation research. Treated as immutable once work begins (except Implementation Plan, which may be refined during execution). - `progress.md` — Hand-maintained status log: execution plan (tracking actual progress), current state, blockers, decisions made during implementation, links to related ADRs or PRs. Updated throughout the feature lifecycle.

Template: README.md

Feature: [Feature Title]

GitHub Issue: #NNN

Status: [Planning | In Progress | Review | Done]

Overview

Brief description of what this feature does and why it matters.

Requirements

- REQ-001: [Functional requirement]
- REQ-002: [Non-functional requirement]
- REQ-003: [Constraint or dependency]

Acceptance Criteria

- [] ACC-001: Verifies REQ-001 – [testable condition]
- [] ACC-002: Verifies REQ-002 – [testable condition]
- [] ACC-003: Verifies REQ-003 – [testable condition]

Scope

What is included in this feature:

- Item 1
- Item 2

What is explicitly out of scope:

- Item A
- Item B

Dependencies

- Depends on: [other feat-NNN-slug, ADR id, or external]
- Blocks: [other feat-NNN-slug]

Design Notes

Any architectural decisions, patterns, or design rationale relevant to this feature.

Related ADRs

- [ADR id]: [Title]
- [ADR id]: [Title]

Implementation Plan

High-level breakdown of work phases and tasks:

Phase 1: [Phase name]

- Task 1.1: [description] – Depends on: [none/other tasks]
- Task 1.2: [description] – Depends on: Task 1.1

Phase 2: [Phase name]

- Task 2.1: [description] – Depends on: Task 1.2
- Task 2.2: [description] – Depends on: Task 2.1

Template: progress.md

```
# Progress: [Feature Title]

## Current Status

**As of [YYYY-MM-DD]**: [Brief status summary]

## Execution Plan

Tracks actual progress against the Implementation Plan in README.md. Update task
status here as work progresses.

### Phase 1: [Phase name]
- [x] Task 1.1: [description] – Completed [YYYY-MM-DD]
- [ ] Task 1.2: [description] – In progress, ETA [YYYY-MM-DD]

### Phase 2: [Phase name]
- [ ] Task 2.1: [description] – Blocked by: [blocker]
- [ ] Task 2.2: [description] – Not started

### Blockers

- [ ] Blocker 1 – [description, impact, mitigation]
- [ ] Blocker 2 – [description, impact, mitigation]

(Remove this section if no blockers.)

## Recent Updates

### [YYYY-MM-DD]
- Completed: [what was done]
- Next: [what comes next]
- Notes: [any relevant context]

### [YYYY-MM-DD]
- Completed: [what was done]
- Next: [what comes next]
- Notes: [any relevant context]

## Decisions Made

- **[YYYY-MM-DD]**: [Decision] – [Rationale]
- **[YYYY-MM-DD]**: [Decision] – [Rationale]

## Related PRs / Commits

- [PR #NNN](link): [description]
- [Commit hash](link): [description]
```

Pros: - Clear separation of concerns: README.md is the immutable "contract" (what we committed to), progress.md is the mutable "journal" (what actually happened) - Auditability: you can see what was promised vs. what was delivered by comparing the two files - Plan stays clean and focused: not

cluttered with progress updates - Easier to reference just the plan without scrolling through progress history - Requirements and acceptance criteria are co-located with clear traceability - Implementation Plan lives in README.md (single source of truth for the plan) - Execution Plan lives in progress.md (single source of truth for actual progress)

Cons: - Two files to maintain — requires reading both to get the full picture - More complex structure - Requires discipline to keep progress.md updated (hand-maintained, not auto-generated) - Agents need to read both files to understand plan + current status - Implementation Plan and Execution Plan are in separate files (requires cross-referencing)

Open Questions: - Not chosen — this option was not carried forward with the same scrutiny/refinement pass as Option 1, since Option 1 was selected as the Decision Outcome. Retained here for reference only. - Still has the original Implementation Plan / Execution Plan split (across two files, no less), i.e. the sync-burden issue identified and resolved in Option 1 via a single Task List with inline status was never addressed here. - No history.md-equivalent or rotation mechanism for progress.md's Recent Updates growth. - Template-location ambiguity applies here too — no separate reusable template files, only what's embedded in this ADR. - To be answered later, if this option is ever revisited: same open items as Option 1 (archival/lifecycle rule, template versioning path, potential atomic status-update tooling).

More Information

- specmgr repository: <https://github.com/anomalyco/biz.dfch.SpecMgr>
-

REQ 678319da-f8e6-4f65-8f98-1096024012af

```
classification: null
created: '2026-09-03 10:26:29.090+02:00'
id: 678319da-f8e6-4f65-8f98-1096024012af
status: draft
type: req
updated: '2026-09-03 10:26:29.090+02:00'
version: 1.0.0
```

Architecture Decision Record Document Management

THE system shall provide Architecture Decision Record (ADR) document management, including create, read, update-frontmatter, update-section, option management, status-change, and validate operations for MADR-style decision records with a Context/Decision Drivers/Considered Options/Decision Outcome/Pros-and-Cons-of-the-Options structure.

Description

AGENTS.md's Status section documents the adr package as the original, most complete domain: 11 @mcp.tool() wrappers (get_adr, list_adr, create_adr, update_frontmatter, update_section, the five option_* tools, and validate_adr), with status changes going through the generic set_status tool (type="adr"). Although ADR is deprecated in favor of dec for new decisions, all 28 existing ADR documents on disk remain status: accepted and continue to be managed through these tools.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the adr bullet) and .specmgr/feat/feat-9-doc-in-specmgr/adr-tool-plan.md.

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ 64065cad-bb84-45c4-9e18-b2a8c5ce6865

```
classification: null
created: '2026-09-03 10:26:32.304+02:00'
id: 64065cad-bb84-45c4-9e18-b2a8c5ce6865
status: draft
type: req
updated: '2026-09-03 10:26:32.304+02:00'
version: 1.0.0
```

Requirement Document Management

THE system shall provide requirement (REQ) document management, including create, read, list, and validate operations for individual requirement statements categorized by ISO/IEC 25010:2023 characteristic, RFC 2119 obligation level, priority, and related-artifact cross-references.

Description

AGENTS.md's Status section documents the req package's tools (`create_req`, `parse_req`, `list_req`, `validate_req`), with whole-body/line-range updates, status changes, classification changes, and deletions dispatched through the generic `update/set_status/set_classification/delete` tools (`type="req"`). The `specmgr://req/schema` shows a REQ document's mandatory `##Characteristics` field must name at least one ISO/IEC 25010:2023 main characteristic.

Characteristics

- 1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the req bullet) and `.specmgr/feat/feat-6-requirement-artifact/README.md`.

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ 594afce9-7166-47b2-8e8f-788b9ed68c8e

```
classification: null
created: '2026-09-03 10:26:35.045+02:00'
id: 594afce9-7166-47b2-8e8f-788b9ed68c8e
status: draft
type: req
```

updated: '2026-09-03 10:26:35.045+02:00'
version: 1.0.0

Use Case Document Management

THE system shall provide use case (UC) document management, including create, read, list, and validate operations for operational-scenario documents, with a raw read mode that returns the frontmatter-stripped body text unchanged for line-range editing.

Description

AGENTS.md's Status section documents the uc package's tools (`create_uc`, `parse_uc`, `list_uc`, `get_uc`, `get_uc_example`, `get_uc_template`, `validate_uc`), mirroring req's shape. Its schema exists in two versions, `uc/models/v1/` (legacy) and `uc/models/v2/` (current), both inside the domain package rather than under top-level `models/`.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the uc bullet) and `.specmgr/feat/feat-4-use-cases/README.md`.

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ c097fcb4-9bbd-41f8-b774-b2afdc8ecb9

```
classification: null
created: '2026-09-03 10:26:37.717+02:00'
id: c097fcb4-9bbd-41f8-b774-b2afdc8ecb9
status: draft
type: req
```

Task List Document Management

THE system shall provide task list (TSK) document management, including create, read, list, and validate operations for implementation checklists derived from other documents, plus a dedicated `implement_task` prompt that builds a `ToDoWrite` list from a task list's items and uses the `question` tool to resolve ambiguity.

Description

AGENTS.md's Status section documents the `tsk` package's tools (`create_tsk`, `parse_tsk`, `list_tsk`, `get_tsk`, `get_tsk_example`, `get_tsk_template`, `validate_tsk`), mirroring req's/uc's shape, plus the `implement_task` prompt that is distinct from every other domain's create/update prompt pair.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the `tsk` bullet).

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ 152d608b-ea4c-463b-8183-33332fb41e50

```
classification: null
created: '2026-09-03 10:26:40.477+02:00'
id: 152d608b-ea4c-463b-8183-33332fb41e50
status: draft
type: req
```

updated: '2026-09-03 10:26:40.477+02:00'
version: 1.0.0

Requirements-Elicitation Question and Answer Document Management

THE system shall provide question-and-answer (QA) document management, including create, read, list, and validate operations for requirements-elicitation interviews structured by ISO/IEC 25010:2023 characteristic category plus an `## Elicitation Context` section.

Description

AGENTS.md's Status section documents the qa package as a single-schema (v2-only) domain: every question/answer category holds zero or more adjacent, un-headed question/answer pairs directly inside a category section, plus an `## Elicitation Context` section between `## General` and `## Functional Suitability`. An earlier v1 schema (one `### {heading}` H3 per pair) was removed entirely once every QA tool/resource/prompt was repointed at v2.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the qa bullet).

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ f4180953-9f1b-45a5-8474-8d15a5872d49

classification: null
created: '2026-09-03 10:27:04.246+02:00'

```
id: f4180953-9f1b-45a5-8474-8d15a5872d49
status: draft
type: req
updated: '2026-09-03 10:27:04.246+02:00'
version: 1.0.0
```

Problem Statement Document Management

THE system shall provide problem statement (PRB) document management, including create, read, list, and validate operations for Six-Sigma-style 5W2H problem statements.

Description

AGENTS.md's Status section documents the prb package's tools (create_prb, parse_prb, list_prb, get_prb, get_prb_example, get_prb_template, validate_prb) plus prb/prompts/ narrated TodoWrite and question-tool-driven 5W2H interview flows.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the prb bullet).

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ 7c0e56e2-3fa5-437e-b886-1be32b142292

```
classification: null
created: '2026-09-03 10:27:06.481+02:00'
id: 7c0e56e2-3fa5-437e-b886-1be32b142292
status: draft
```

```
type: req
updated: '2026-09-03 10:27:06.481+02:00'
version: 1.0.0
```

Goal Document Management

THE system shall provide goal (GOL) document management, including create, read, list, and validate operations for high-level business goal documents that sit above individual requirements, whose body mirrors REQ minus the ## Characteristics and ## Level sections.

Description

AGENTS.md's Status section documents the gol package's tools (`create_gol`, `parse_gol`, `list_gol`, `get_gol`, `get_gol_example`, `get_gol_template`, `validate_gol`), with `create_gol` first checking `list_gol` for a near-duplicate goal. See `.specmgr/feat/feat-18-goal/README.md` for the full body-shape rationale.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the gol bullet) and `.specmgr/feat/feat-18-goal/README.md`.

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ bb018715-f9e6-4ae6-830c-58e40162ac70

```
classification: null
created: '2026-09-03 10:27:09.701+02:00'
id: bb018715-f9e6-4ae6-830c-58e40162ac70
status: draft
```

```
type: req
updated: '2026-09-03 10:27:09.701+02:00'
version: 1.0.0
```

Risk Register Document Management

THE system shall provide risk (RSK) document management, including create, read, list, and validate operations for risk-register entries with a 5x5 probability/impact assessment both before and after mitigation and a closed TARA response strategy.

Description

AGENTS.md's Status section documents the `rsk` package's tools (`parse_rsk`, `get_rsk`, `list_rsk`, `get_rsk_example`, `get_rsk_template`, `create_rsk`, `validate_rsk`) plus two static domain-knowledge resources, `specmgr://rsk/tara` and `specmgr://rsk/risk-matrix`, that document the closed four-value TARA vocabulary (transfer/accept/reduce/avoid) and the 5x5 scale respectively.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the `rsk` bullet).

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ 1b6975fb-f5c2-4a16-b9db-9f026b8e6912

```
classification: null
created: '2026-09-03 10:27:12.539+02:00'
id: 1b6975fb-f5c2-4a16-b9db-9f026b8e6912
status: draft
```

```
type: req
updated: '2026-09-03 10:27:12.539+02:00'
version: 1.0.0
```

General Decision Document Management

THE system shall provide decision (DEC) document management, including create, read, list, and validate operations for MADR-style decisions that are not architecture-specific, built on the generic `models/md` parser with a simple, renderer-free surface.

Description

AGENTS.md's Status section documents the dec package's tools (`parse_dec`, `get_dec`, `list_dec`, `get_dec_example`, `get_dec_template`, `create_dec`, `validate_dec`). A DEC keeps the ADR's general MADR-style structure (headings, `Options` collection) but has no fine-grained mutation tools or `renderer`: writes persist the caller's raw validated body byte-for-byte.

Characteristics

1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the dec bullet) and `.specmgr/feat/feat-21-decision/README.md`.

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ ccbf7ade-7d9e-4b2e-9868-0740bdc0e824

```
classification: null
created: '2026-09-03 10:27:15.700+02:00'
id: ccbf7ade-7d9e-4b2e-9868-0740bdc0e824
status: draft
```

type: req
updated: '2026-09-03 10:27:15.700+02:00'
version: 1.0.0

Feature Folder Document Management

THE system shall provide feature (FEAT) document management, including create, read, list, and validate operations for the `.specmgr/feat/<id>/README.md` feature-folder convention, plus a dedicated `set_feat_id` tool for renaming a feature's chosen id after the fact.

Description

AGENTS.md's Status section documents the feat package as the one domain whose addressing genuinely deviates from every other domain's precedent (ADR 8cf940c5-3100-485c-a12d-14b59b631712): `id` is a chosen `feat-NNN-slug`, the containing folder's own name, not a server-generated UUID, and documents live one-per-folder as `<base>/<id>/README.md`. This very feature (`feat-84-specmgr-sysrs`) is itself a FEAT document managed through these tools.

Characteristics

- 1. Functional Suitability

Level

MUST

Source

AGENTS.md's Status section (the feat bullet) and `.specmgr/feat/feat-31-feature/README.md`.

Related Artifacts

Goals

- GOL-08666592-a2d2-4309-95c6-3c94248ca342: AI-Agent-Native Specification Artifact Management

REQ 3bbe6a0e-038c-4abb-987c-79d4db8abd51

classification: null
created: '2026-09-03 10:27:42.023+02:00'
id: 3bbe6a0e-038c-4abb-987c-79d4db8abd51

```
status: draft
type: req
updated: '2026-09-03 10:27:42.023+02:00'
version: 1.0.0
```

Standard Operating Procedure Document Management

THE system shall provide Standard Operating Procedure (SOP) document management, including create, read, list, and validate operations for structured, step-by-step operational documents with a RASCI-style responsibility assignment and a closed approval/effectivity lifecycle, built dispatch-only on the generic update/set_status/set_classification/delete tools from day one instead of adding new per-domain tools.

Description

AGENTS.md's Status section documents sop as the first domain built dispatch-only from day one (ADR 36905d5b-8057-4294-8665-c7eed5534db0): it has no per-domain update_sop/set_status_sop/set_classification_sop tools at all, relying entirely on the generic dispatch tools with type="sop". This reduces the number of tools that must be built, tested, and maintained as new domains are added, improving the system's overall modifiability and reducing duplicated logic across domains.

Characteristics

1. Maintainability

Level

MUST

Source

AGENTS.md's Status section (the sop bullet) and ADR 36905d5b-8057-4294-8665-c7eed5534db0.

Related Artifacts

Goals

- GOL-b663528e-08c5-426b-9f20-32192c0a3bdb: Cross-Referenceable, Non-Duplicating Specification Artifacts
-

REQ 10b78b36-abad-4bfe-9281-f75677ff7d09

```
classification: null
created: '2026-09-03 10:27:45.937+02:00'
id: 10b78b36-abad-4bfe-9281-f75677ff7d09
status: draft
type: req
updated: '2026-09-03 10:27:45.937+02:00'
version: 1.0.0
```

Verification Case Record Document Management

THE system shall provide verification case record (VCR) document management, including create, read, list, and validate operations for DTAIS-classified acceptance criteria that record how a single requirement or use case is verified, so that a requirement's fulfillment can be objectively assessed and tested.

Description

AGENTS.md's Status section documents the vcr package's tools (create_vcr, parse_vcr, list_vcr, get_vcr, get_vcr_example, get_vcr_template, validate_vcr). Each `### AC-NNN (Method): ...` entry uses a closed DTAIS vocabulary (Demonstration, Test, Analysis, Inspection, Special, documented by the cross-cutting `specmgr://dtais` resource) that lets an objective, feasible test be designed to determine whether a requirement is met -- the Testability sub-characteristic of Maintainability.

Characteristics

1. Maintainability

Level

MUST

Source

AGENTS.md's Status section (the vcr bullet) and `.specmgr/feat/feat-33-vcr/README.md`.

Related Artifacts

Goals

- GOL-b663528e-08c5-426b-9f20-32192c0a3bdb: Cross-Referenceable, Non-Duplicating Specification Artifacts

REQ 26c37265-1a85-4b18-aada-c9e3db9574a8

```
classification: null
created: '2026-09-03 10:27:50.218+02:00'
id: 26c37265-1a85-4b18-aada-c9e3db9574a8
status: draft
type: req
updated: '2026-09-03 10:27:50.218+02:00'
version: 1.0.0
```

System Requirements Specification Aggregator Document Management

THE system shall provide System Requirements Specification (SYSRS) document management, including create, read, list, and validate operations for an aggregator document type that ties together existing gol/prb/qa/uc/req/rsk/dec/adr/vcr artifacts into one coherent, navigable specification via type-tagged cross-reference lists, built dispatch-only on the generic update/set_status/set_classification/delete tools from day one.

Description

AGENTS.md's Status section documents sysrs alongside sop/vcr as built dispatch-only from day one (ADR 36905d5b-8057-4294-8665-c7eed5534db0): 7 tools (create_sysrs, parse_sysrs, list_sysrs, get_sysrs, get_sysrs_example, get_sysrs_template, validate_sysrs), no per-domain update_sysrs/set_status_sysrs tools of its own. Its sections accept only <TYPE> <uuid>: <title> cross-reference bullets, never inline copies of the referenced document's content, so a SysRS never duplicates the requirements/goals/decisions it aggregates.

Characteristics

1. Maintainability

Level

MUST

Source

AGENTS.md's Status section (the sysrs bullet) and .specmgr/feat/feat-32-sysrs/README.md.

Related Artifacts

Goals

- GOL-b663528e-08c5-426b-9f20-32192c0a3bdb: Cross-Referenceable, Non-Duplicating Specification Artifacts

REQ bad7e9c7-f794-477b-b64f-ce04645c6ef3

```
classification: null
created: '2026-09-03 10:27:53.793+02:00'
id: bad7e9c7-f794-477b-b64f-ce04645c6ef3
status: draft
type: req
updated: '2026-09-03 10:27:53.793+02:00'
version: 1.0.0
```

Generic Cross-Domain Document Dispatch Tools

THE system shall provide generic, type-dispatched update, set_status, set_classification, and delete tools that operate uniformly across every whole-body document domain, so that adding a new document type requires only one dispatch entry per generic tool, not new per-domain update_<d>/set_status_<d>/set_classification_<d>/delete_<d> tools.

Description

AGENTS.md's Status section documents the cross-cutting general package: the generic update tool (whole-body and line-range replace for the twelve whole-body domains), set_status (all thirteen domains including adr), set_classification, and delete, plus mdformat and shared resources (specmgr://version, specmgr://iso25010, specmgr://dtais, specmgr://rasci). ADR 36905d5b-8057-4294-8665-c7eed5534db0 fixes this as the required convention for every future domain (e.g. the reserved but not-yet-built ac domain), reducing per-domain tool duplication and the maintenance burden of the codebase as a whole.

Characteristics

1. Maintainability

Level

MUST

Source

AGENTS.md's Status section (the general bullet) and ADR 36905d5b-8057-4294-8665-c7eed5534db0.

Related Artifacts

Goals

- GOL-b663528e-08c5-426b-9f20-32192c0a3bdb: Cross-Referenceable, Non-Duplicating Specification Artifacts